A system for analyzing historic network traffic with current rulesets

Kyle Koza
kyle.koza@security.gatech.edu

Abstract—Signature based intrusion detection systems (IDS) are widely used by security operations staff in order to detect and potentially block intrusion attempts. However, these systems are only as good as the signatures they run. Unfortunately, signatures can only be developed after the discovery of vulnerabilities and exploits. It is also uncertain if any groups were using these previously undiscovered exploits prior to their public discovery. This leaves a gap in effectiveness between the time of exploit development (whether public or private) and the time that signatures are published. In this paper, I produced a proof of concept system that will run current signatures over historic network traffic in order to eliminate a security blind spot. The data produced by this system can also be used as a measure of effectiveness of the various signature sets.

1 Introduction

1.1 Signature based intrusion detection systems

Signature based intrusion detection systems (IDS) are widely used by security operations staff in order to detect and potentially block intrusion attempts. These systems run network traffic through a set of signatures and output alerts based on which signatures match the network traffic. Like signature based malware detection, signature based intrusion detection systems are only as effective as the signatures they use. Since signatures only look for traffic that matches previously seen malicious traffic, signature based intrusion detection systems are not very good at identifying previously unseen malicious traffic.

Once a vulnerability is discovered, signatures can be developed to detect attacks and exploit against the vulnerability. However, it takes time to write and publish signatures. Unless a vulnerability is released to a signature writing team prior to being released, it is very likely that malicious users will be exploiting vulnerabilities before we have signatures to detect the malicious traffic.

1.2 Heartbleed

Heartbleed, a serious OpenSSL bug dating back to December 31, 2011, was discovered by a Google employee on or just prior to March 21st 2015. On the evening of March 21st, Google employees created a patch which is then applied to Googles services. Ten days later (March 31st), CloudFlare is made aware of the bug and applies the patch to their services. On April 1st, Google notified the OpenSSL team and plans are made to publish the patch on April 9th. The release was rushed for April 7th because another security research lab independently discovers the flaw [1]. Signatures for the exploitation of Heartbleed were published the next day on April 8th, which began firing alerts immediately in many environments [2], [3]. In the case of Heartbleed, at least 17 days passed between the discovery of the vulnerability and a publicly released patch and an additional day until an IDS signature was released to the public. The bug was present in the code for 828 days prior to a usable IDS signature.

It is also entirely possible that other groups may have been exploiting the vulnerability prior to its discovery by Google. In fact, several sources claim to have seen the exploit in their logs in the months prior to the public release, though the evidence in these cases are not conclusive [4]. Everyone running a vulnerable version of OpenSSL was advised to replace their SSL keys and certificates and users of services with vulnerable versions of OpenSSL were advised to change their passwords. Due to the way the exploit works it was possible someone could steal information from a server leaving little to no trace of the attack. Without evidence to the contrary, administrators and users must operate under the assumption that they have been compromised. With this problem in mind, for my practicum, I created a system that allows an administrator to run up to date IDS signatures over historic network traffic. The system then presents the results in a way that can be as actionable as a normal IDS.

2 The System

2.1 The Idea

As far as I can tell, no one is currently running current IDS signatures over past traffic. This is potentially because of the cost of storing full packet capture data. It may be feasible for small organizations to store full packet data due to the limited amount of network traffic they see, however, large organizations, such as

Georgia Tech, regularly see terabytes of traffic per day. The expense to keep such a large amount of data over an extended period of time would be substantial. A solution is to keep a sample of traffic each day, possibly in the same manner a backup system would: full packet captures for several days and then decreasing percentages of traffic for older traffic. While this would not give an administrator a definitive answer regarding the exploitation of his systems, it could give him some confidence either way. This data is also interesting from a research perspective because it would give us the ability to confirm report of groups exploiting previously undiscovered vulnerabilities.

2.2 Statistics regarding the rulesets

Another benefit this system can provide is detailed statistics about the rulesets. We can get an idea about how long it takes for each signature writing team to produce signatures for exploits, by comparing how much time between when we see alerts fire until the signature is published. We may also be able to draw other insights from the data about the effectiveness of the rulesets and whether it is worth paying for the professional versions of the rulesets compared to the free versions.

2.3 The Process

Georgia Tech CyberSecurity recently implemented a device capable of storing several days of full network traffic, which I utilized for my research. I archived a portion of each days traffic to use as a proof of concept for this project.

First, the program downloads and stores the updated rulesets (bundles of signatures). Which ruleset(s) to use can be specified in the configuration for the system. For the purposes of my practicum, I have chosen to use three rulesets: Snorts Talos, Emerging Threats Pro, and the community version of Emerging Threats.

The program then configures the IDS engine to use one of the rulesets. The proof of concept uses Suricata, due to the ease of configuring it programatically. The IDS is then run over the archived packet captures and the alerts generated by the rulesets are stored. The program repeats this process for each ruleset.

The alerts are then parsed into Splunk and stored with information that specifies the date and name of the ruleset that generated the alerts. Using several queries I developed, Splunk then analyzes the alerts to determine if any of the alerts are previously unseen.

3 Challenges

The first challenge I faced while completing this project was Georgia Tech CyberSecuritys packet capture solution. It took several weeks to work out several bugs that prevented its use. Storage and processing limitations were also a factor due to the workstation I was using for this project. This limited me to 5 minute samples for packet capture data.

Another substantial challenge was making the IDS engines work in a way that they were not designed for. IDS engines were designed to run in a single ruleset configuration, whether that includes a single ruleset or multiple rulesets. The rulesets are updated and signatures are added and removed, but the rulesets stay the same. I began by using Snort and found that managing the rulesets was going to be a challenge based on the way the configuration is performed. For ease, I switched to Suricata which uses YAML for its configuration which is easy to parse and modify programmatically.

Originally, I had intended to simply diff the outputs of the alerts each day to check for differences, but this proved to be challenging because alerts were sometimes fired out of order. I believe this was happening because Suricata is multithreaded. This problem prompted me to look into alternative ways to parse and compare the alerts. I settled on Splunk because it is very good at analyzing large sets of data. I wrote Splunk field extractions, shown below, to correctly parse fields from the Suricata fast alerts log and used field transforms to parse out the relevant dates from the alert file path.

```
\[\d+:(?<sid>\d+):\d\]\s(?<signature>.*?)(\s\[).*\[
    Classification:\s(?<category>.*?)\]
```

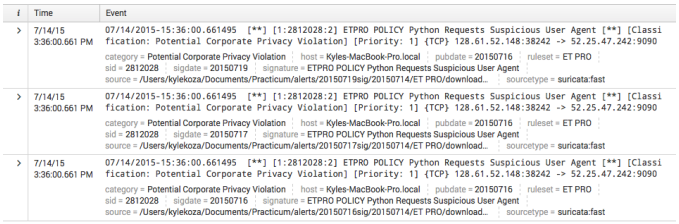
4 Results

I completed the proof of concept version of the system in early July. It needs some additional work to get it into a state ready to release to the public, but it is fully functional. It its current form, it is able to download and manage the rulesets, run the IDS engine over the packet captures, and output results in a way that can be analyzed.

I ran the system using both the community and professional signatures from Emerging Threats and also the Snort VRT signatures. The signatures spanned dates between mid-June and mid-July 2015. I retained packet captures for 7 days spanning early to mid July. Most of the packet captures were 5 minutes long with the start time chosen at random with the following Python command:

```
>>> print str(random.randint(0,23)) + ":" + str(
    random.randint(0,59))
```

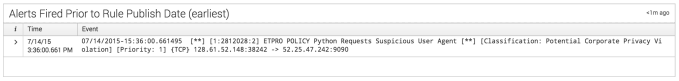
Running the tool with these settings returned one signature that fired on traffic that occurred previous to the date which the signature was written.



i	Time	Event
>	7/14/15 3:36:00.661 PM	07/14/2015-15:36:00.661495 [**] [1:2812028:2] ETPRO POLICY Python Requests Suspicious User Agent [**] [Classification: Potential Corporate Privacy Violation] [Priority: 1] {TCP} 128.61.52.148:38242 -> 52.25.47.242:9090 category = Potential Corporate Privacy Violation host = Kyles-MacBook-Pro.local pubdate = 20150716 ruleset = ET PRO sid = 2812028 sigdate = 20150719 signature = ETPRO POLICY Python Requests Suspicious User Agent source = /Users/kylekoza/Documents/Practicum/alerts/20150719sig/20150714/ET PRO/download... sourcetype = suricata:fast
>	7/14/15 3:36:00.661 PM	07/14/2015-15:36:00.661495 [**] [1:2812028:2] ETPRO POLICY Python Requests Suspicious User Agent [**] [Classification: Potential Corporate Privacy Violation] [Priority: 1] {TCP} 128.61.52.148:38242 -> 52.25.47.242:9090 category = Potential Corporate Privacy Violation host = Kyles-MacBook-Pro.local pubdate = 20150716 ruleset = ET PRO sid = 2812028 sigdate = 20150717 signature = ETPRO POLICY Python Requests Suspicious User Agent source = /Users/kylekoza/Documents/Practicum/alerts/20150717sig/20150714/ET PRO/download... sourcetype = suricata:fast
>	7/14/15 3:36:00.661 PM	07/14/2015-15:36:00.661495 [**] [1:2812028:2] ETPRO POLICY Python Requests Suspicious User Agent [**] [Classification: Potential Corporate Privacy Violation] [Priority: 1] {TCP} 128.61.52.148:38242 -> 52.25.47.242:9090 category = Potential Corporate Privacy Violation host = Kyles-MacBook-Pro.local pubdate = 20150716 ruleset = ET PRO sid = 2812028 sigdate = 20150716 signature = ETPRO POLICY Python Requests Suspicious User Agent source = /Users/kylekoza/Documents/Practicum/alerts/20150716sig/20150714/ET PRO/download... sourcetype = suricata:fast

Fig. 1. Signatures fired on traffic prior to being published

The alert fired three times due to its inclusion in the signatures for July 16th, 17th, and 19th. I did not expect this behavior and believe it will cause a problem when trying to operationalize the system, because a security operations team would not want to see the same alert repeatedly especially if they have already handled it. To mitigate this problem, I created a Splunk query that will only display an alert the first time the alert is fired. This could be easily run as an alert to generate a security event for security operations staff to research.

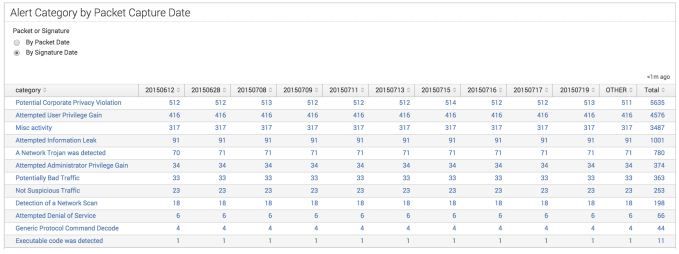


Alerts Fired Prior to Rule Publish Date (earliest) <1m ago

i	Time	Event
>	7/14/15 3:36:00.661 PM	07/14/2015-15:36:00.661495 [**] [1:2812028:2] ETPRO POLICY Python Requests Suspicious User Agent [**] [Classification: Potential Corporate Privacy Violation] [Priority: 1] {TCP} 128.61.52.148:38242 -> 52.25.47.242:9090

Fig. 2. An example of an alert fired for security operations staff

The rulesets were all fairly consistent over the sample set, generally the same rules fired each day. Several rules fired inconsistently, namely "ET POLICY Protocol 41 IPv6 encapsulation potential 6in4 IPv6 tunnel active." This signature caused the majority of the fluctuation between count of alerts by signature date. Even running the same signature set over the same pcaps returned inconsistency when firing this rule. This will require more research on my part, but may prove to be a bug in Suricata.



Alert Category by Packet Capture Date

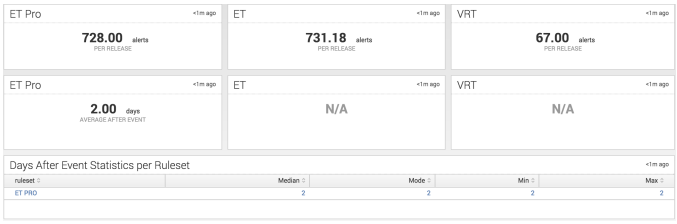
Packet or Signature
- By Packet Date
- By Signature Date

<1m ago

category	20150612	20150628	20150708	20150709	20150711	20150713	20150715	20150716	20150717	20150719	OTHER	Total
Potential Corporate Privacy Violation	512	512	513	512	512	512	514	512	512	513	511	5635
Attempted User Privilege Gain	416	416	416	416	416	416	416	416	416	416	416	4576
Misc activity	317	317	317	317	317	317	317	317	317	317	317	3487
Attempted Information Leak	91	91	91	91	91	91	91	91	91	91	91	1001
A Network Trojan was detected	70	71	71	71	71	71	71	71	71	71	71	780
Attempted Administrator Privilege Gain	34	34	34	34	34	34	34	34	34	34	34	374
Potentially Bad Traffic	33	33	33	33	33	33	33	33	33	33	33	363
Not Suspicious Traffic	23	23	23	23	23	23	23	23	23	23	23	253
Detection of a Network Scan	18	18	18	18	18	18	18	18	18	18	18	198
Attempted Denial of Service	6	6	6	6	6	6	6	6	6	6	6	66
Generic Protocol Command Decode	4	4	4	4	4	4	4	4	4	4	4	44
Executable code was detected	1	1	1	1	1	1	1	1	1	1	1	11

Fig. 3. Chart of alerts by category and signature date over all packet capture dates

The community version of Emerging Threats (ET) fired more alerts per day on average then the professional version. This was also because of the strangeness regarding the IPv6 encapsulation signature. The VRT ruleset fired much less alerts than the Emerging Threats rulesets. This seems to be because of a lack of the "information" alerts included with ET. Normally, an IDS administrator would disable many of the ET signatures in their installation. For example, at Georgia Tech we disable the majority of the Policy signatures for campus, since we do not necessarily care to see alerts on Dropbox usage etc.



ET Pro <1m ago: 728.00 alerts PER RELEASE
ET <1m ago: 731.18 alerts PER RELEASE
VRT <1m ago: 67.00 alerts PER RELEASE
ET Pro <1m ago: 2.00 days AVERAGE AFTER EVENT
ET <1m ago: N/A
VRT <1m ago: N/A

Days After Event Statistics per Ruleset <1m ago

ruleset	Median	Mode	Min	Max
ET PRO	2	2	2	2

Fig. 4. Detailed statistics comparing each ruleset

```
index=main
| lookup sigdates sid as sid
| eval traffic_date = strftime(_time,"%Y%m%d")
| search pubdate=* traffic_date<pubdate
| eventstats min(sigdate) as first_seen by _time,sid
| where sigdate=first_seen
```

I created two Splunk dashboards to assist in analyzing the data. One of the dashboards, as seen in the appendix, was created with security operations staff in mind as a way to compare the outputs of the different rulesets and packet capture dates. The other dashboard, also included in the appendix, is focused more on analysis of the rulesets. With limited data, this dashboard does not give us any insights, but with a larger data set, I think this dashboard will become very valuable.

4.1 Splunk queries

The following Splunk query lists the mean amount of signatures fired by the specified ruleset each day. The ruleset variable can be changed to whichever ruleset you are interested in. In the Signature Statistics dashboard, I have created dashboard panels for each of the rulesets.

```
index=main ruleset="ET_PRO" signature!="SURICATA*"
| stats count by sigdate
| stats avg(count) as Mean
| eval Mean = round(Mean, 2)
```

This Splunk query lists common statistics about each of the rulesets: the median, mode, min, and max number of alerts fired by each ruleset per day.

```
index=main
| lookup sigdates sid as sid
| eval traffic_date = strftime(_time,"%Y%m%d")
| search pubdate=* traffic_date<pubdate
| eventstats min(sigdate) as first_seen by _time,sid
| where sigdate=first_seen
| eval time_diff = pubdate-traffic_date
| stats median(time_diff) as Median mode(time_diff)
    as Mode min(time_diff) as Min max(time_diff) as
    Max by ruleset
```

This Splunk query gives the average number of days between the first time network traffic occurred and the date that a signature was published that alerted on it.

This can also be customized for the ruleset by changing the ruleset variable in the query. In the Signature Statistics dashboard, I have created a box for each of the rulesets to quickly compare this number across the rulesets. Since I only had one alert fire for traffic that occurred before the signatures publishing date, only the ET Pro box has statistics.

```
index=main
| lookup sigdates sid as sid
| eval traffic_date = strftime(_time,"%Y%m%d")
| search pubdate=* traffic_date<pubdate
| eventstats min(sigdate) as first_seen by _time,
    asid
| where sigdate=first_seen
| eval time_diff = pubdate-traffic_date
| where ruleset="ET_PRO"
| stats avg(time_diff) as Mean
| eval Mean = round(Mean, 2)
```

The following Splunk query is the first version of the query listed in the Results section. This query returns each alert that fired for traffic prior to the signature publishing date. This will fire multiple times if the signature is included in future releases.

```
index=main
| lookup sigdates sid as sid
| eval traffic_date = strftime(_time,"%Y%m%d")
| search pubdate=* traffic_date<pubdate
```

This Splunk query makes use of input boxes on the dashboard that allow you to specify the date of the packet capture and the ruleset to be examined. It will then return a chart of the count of alerts per signature date.

```
index=main pcapdate="$pcapdate$" ruleset="$ruleset$"
     signature!="SURICATA*"
| chart count by sigdate
```

This query allows you to toggle between displaying a chart of the count of rules fired per category and signature date and a chart of the count of rules fired per category and the packet capture date.

```
index=main signature!="SURICATA*" pcapdate=*
| chart count by category,$field$
| addtotals
| sort -Total
```

Lastly, this Splunk query allows security operations staff to dive into the differences between each result set. You would choose a packet capture date and then the two signature dates you are interested in finding the differences between. You can also choose one or all of the rulesets to compare. The query will return any alerts that fired for the second signature date that did not fire on the first signature date.

```
index=main pcapdate=$pcap$ sigdate=$sigdate2$
    ruleset="$ruleset1$" signature!="SURICATA*" NOT
    [search index=main pcapdate=$pcap$ sigdate=
    $sigdate1$ ruleset="$ruleset1$" signature!="
    SURICATA*"
| fields - _* source]
```

5 Future Work

I would like to continue development of this system and eventually publish it as open source software for the use of security teams. There are a few improvements I would like to make in the programming before release including:

- Programming the system to only download rulesets that have had updates
- Creating the logic to allow the system to only run newly published signatures over the past packet captures. This would significantly increase the processing speed of the IDS engine. Running unchanged signatures over the past packet captures will just produce redundant results.
- More configuration options including how and where to store the alerts

Also, running the system over more data to get additional insights is critical. I will need to obtain systems with processing power and storage in order to produce meaningful data.

References

[1] B. Grubb, "Heartbleed disclosure timeline: who knew what and when," 2014. [Online]. Available: http://www.smh.com.au/it-pro/security-it/heartbleed-disclosure-timeline-who-knew-what-and-when-20140415-zqurk.html

[2] F. Trudeau, "Emergency heartbleed signatures," 2014. [Online]. Available: https://lists.emergingthreats.net/pipermail/emerging-sigs/2014-April/024058.html

[3] B. Stultz, "Heartbleed memory disclosure - upgrade openssl now!" 2014. [Online]. Available: http://vrt-blog.snort.org/2014/04/heartbleed-memory-disclosure-upgrade.html

[4] P. Eckersley, "Wild at heart: Were intelligence agencies using heartbleed in november 2013?" 2014. [Online]. Available: https://www.eff.org/deeplinks/2014/04/wild-heart-were-intelligence-agencies-using-heartbleed-november-2013

6 Appendix

Listing 1. A Python script that manages the configuration of Suricata's rulesets

```
import os
import yaml

def getRuleFiles(path):
    # List all files contained in the given
        directory that have a .rules
    # extension
    result = []
    for file_ in os.listdir(path):
        if file_.endswith(".rules"):
            result.append(file_)
    return result

def getRules(configFile):
    # Get the currently enabled rules from the
        specified config file
    with open(configFile, 'r') as f:
        config = yaml.load(f)
        return config["rule-files"]
```

```
def clearRules(configFile):
    # Clear rules from configFile
    with open(configFile, 'r+b') as f:
        config = yaml.load(f)
        config["rule-files"] = []
        config["et-rule-files"] = []
        f.seek(0)
        yaml.dump(config, f, explicit_start=True,
            version=(1,1), default_flow_style=False,
             indent=True)
        f.truncate()
        f.close()

def addRules(rules, rulePath, configFile):
    # Add the list of rules to the config file
        specified
    with open(configFile, 'r+b') as f:
        config = yaml.load(f)
        config["rule-files"].extend(rules)
        config["default-rule-path"] = rulePath
        f.seek(0)
        yaml.dump(config, f, explicit_start=True,
            version=(1,1), default_flow_style=False)
        f.truncate()
        f.close()

def loadRules(date, ruleset, configFile):
    import defaults
    clearRules(configFile)
    rulePath = os.path.normpath(os.path.join(
        defaults.getDefaultRulesPath(),
                            date + "-" + ruleset, "
                                rules"))
    addRules(getRuleFiles(rulePath),
             rulePath, configFile)
```

Listing 2. A simple Python script to run Suricata with the correct options

```
import suriConf
import pipes
import subprocess

def runSuricata(configFile, pcapFile, alertDir):
    subprocess.call(["suricata", "-c", configFile, "
        -r",
                     pcapFile, "-l", alertDir])
```

Listing 3. The main Python script which runs all of the various pieces

```
import os
import datetime
import argparse
import defaults
import ConfigParser

DATE = datetime.date.today().strftime("%Y%m%d")

parser = argparse.ArgumentParser(description='Run
    the process')

group = parser.add_mutually_exclusive_group()
group.add_argument('--pcap', '-p', help="A pcap file
    to scan")
group.add_argument('--pcapdate', '-P', help="The
    date of pcaps to scan or all\
                                                to scan
    over all pcaps")
parser.add_argument('--date', '-d', help="The date
    of the signatures to run",
                    default=DATE)
parser.add_argument('--ruleset', '-r', help="The
    ruleset to use")

parser.add_argument('--no-download-rules', help="
    Download the days rules",
                    action='store_false', dest='
                        download')

parser.add_argument('--run-all', help="Run all
    signatures over all pcaps",
                    action='store_true', dest='runall
                        ')

parser.set_defaults(download=True, runall=False)

def main(download=True, pcap=None, pcapdate=None,
    sigdate=DATE, ruleset=None):
    config = ConfigParser.RawConfigParser()
    config.read(defaults.getDefaultRulesConfig())

    if download:
        import downloadRules
        downloadRules.downloadRules() #TODO: add
            config argument

    if pcapdate is None:
        pcapfolder = os.path.normpath(os.path.join(
            defaults.getDefaultPcapFolder(), DATE))
    else:
        pcapfolder = os.path.normpath(os.path.join(
            defaults.getDefaultPcapFolder(),
            pcapdate))

    import suriConf
    for section in config.sections():
        import suricata
        suriConf.loadRules(sigdate, section,
            defaults.getDefaultSuricataConfig())

        pcaps = []

        if pcapdate == "all":
            import re
            pcapfolder = defaults.
                getDefaultPcapFolder()
            for folder in os.listdir(pcapfolder):
                if os.path.isdir(os.path.join(
                    pcapfolder, folder)):
                    match = re.search("^\d+", folder
                        )
                    if match is not None:
                        for file_ in os.listdir(os.
                            path.join(pcapfolder,
                            folder)):
                            pcaps.append(os.path.
                                normpath(os.path.
                                join(pcapfolder,

        else:
            for file_ in os.listdir(pcapfolder):
```

```
                if file_.endswith((".pcap",)):
                    pcaps.append(os.path.normpath(os
                        .path.join(pcapfolder, file_
                        )))

        for pcap in pcaps:
            path, file_ = os.path.split(pcap)
            pcapDate = path.split(os.sep)[-1]
            outdir = os.path.normpath(os.path.join(
                defaults.getDefaultAlertsDir(),
                sigdate + "sig",
                                pcapDate, section,
                                    file_))
            if not os.path.exists(outdir):
                os.makedirs(outdir)
            suricata.runSuricata(defaults.
                getDefaultSuricataConfig(), pcap,
                outdir)

def runall():
    import re

    pcapdates = []
    ruledates = []

    pcapdir = defaults.getDefaultPcapFolder()
    sigdir = defaults.getDefaultRulesPath()

    for folder in os.listdir(pcapdir):
        if os.path.isdir(os.path.join(pcapdir,
            folder)):
            pcapdates.append(folder)

    for folder in os.listdir(sigdir):
        if os.path.isdir(os.path.join(sigdir, folder
            )):
            match = re.search("^\d+", folder)
            if match is not None:
                if match.group(0) not in ruledates:
                    ruledates.append(match.group(0))

    for sigdate in ruledates:
        for pcapdate in pcapdates:
            main(sigdate=sigdate, download=False,
                pcapdate=pcapdate)


if __name__ == "__main__":
    args = parser.parse_args()
    args = vars(args)
    if args["runall"]:
        runall()
    else:
        main(sigdate=args["date"], download=args['
            download'], pcap=args['pcap'],
             pcapdate=args['pcapdate'])
```

Listing 4. A script to download the various signatures specified in the configuration

```
import argparse

parser = argparse.ArgumentParser(description='
    Download␣Suricata␣rules␣specified.')
parser.add_argument('--config', '-c',
                    help='Path␣to␣the␣rules␣
                        configuration␣file', default
                        =None)
parser.add_argument('--outputPath', '-o',
                    help='Path␣to␣rules␣output␣
                        directory', default=None)
```

```
def downloadRules(configfile=None, outpath=None):
    import os
    import datetime
    import tarfile
    import tempfile
    import requests
    import ConfigParser

    config = ConfigParser.RawConfigParser()

    basepath = os.path.dirname(os.path.realpath(
        __file__))
    if configfile == None:
        configfile = os.path.join(basepath, "..", "
            conf", "rules.conf")
    if outpath == None:
        outpath = os.path.join(basepath, "..", "
            rules")

    config.read(configfile)
    DATE = datetime.date.today().strftime("%Y%m%d")

    rules = {}
    for section in config.sections():
        rules[section] = config.get(section, "url")

    for ruleset,url in rules.iteritems():
        with tempfile.TemporaryFile() as handle:
            response = requests.get(url, stream=True
                , verify=True)

            if not response.ok:
                pass

            for block in response.iter_content(1024)
                :
                if not block:
                    break

                handle.write(block)

            handle.seek(0)
            tar = tarfile.open(fileobj=handle, mode=
                'r:gz')
            tar.extractall(path=os.path.join(outpath
                , DATE+"-"+ruleset))

if __name__ == "__main__":
    args = parser.parse_args()
    args = vars(args)

    downloadRules(configfile=args["config"], outpath
        =args["outputPath"])
```

Listing 5. A script that determines where the default locations. This part will need to be expanded to take into account user defined settings

```
import os

BASEPATH = os.path.dirname(os.path.realpath(__file__
    ))

def getDefaultRulesConfig():
    configfile = os.path.join(BASEPATH, "..", "conf"
        , "rules.conf")
    return os.path.normpath(configfile)

def getDefaultSuricataConfig():
```

```
    configfile = os.path.join(BASEPATH, "..", "conf"
        , "suricata.yaml")
    return os.path.normpath(configfile)

def getDefaultRulesPath():
    rulePath = os.path.join(BASEPATH, "..", "rules")
    return os.path.normpath(rulePath)

def getDefaultAlertsDir():
    alertsdir = os.path.join(BASEPATH, "..", "alerts
        ")
    return os.path.normpath(alertsdir)

def getDefaultPcapFolder():
    pcapfolder = os.path.join(BASEPATH, "..", "pcaps
        ")
    return os.path.normpath(pcapfolder)
```

Signature Statistics

ET Pro <1m ago

728.00 alerts
PER RELEASE

ET <1m ago

731.18 alerts
PER RELEASE

VRT <1m ago

67.00 alerts
PER RELEASE

ET Pro <1m ago

2.00 days
AVERAGE AFTER EVENT

ET <1m ago

N/A

VRT <1m ago

N/A

Days After Event Statistics per Ruleset <1m ago

ruleset	Median	Mode	Min	Max
ET PRO	2	2	2	2

Alerts Fired Prior to Rule Publish Date (earliest) <1m ago

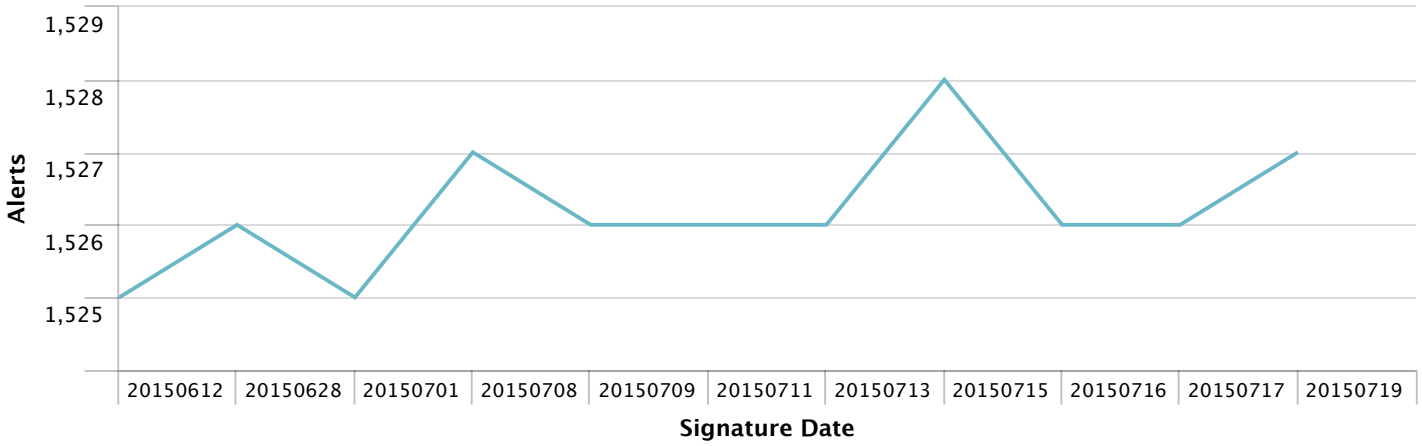
Time	Event
7/14/15 3:36:00.661 PM	07/14/2015-15:36:00.661495 [**] [1:2812028:2] ETPRO POLICY Python Requests Suspicious User Agent [**] [Classification: Potential Corporate Privacy Violation] [Priority: 1] {TCP} 128.61.52.148:38242 -> 52.25.47.242:9090

Historic IDS

Packet Capture Date *

Ruleset *

2m ago



Alert Category by Packet Capture Date

Packet or Signature

- By Packet Date
- By Signature Date

2m ago

category	20150612	20150628	20150708	20150709	20150711	20150713	20150715	20150716	20150717	20150719	OTHER	Total
Potential Corporate Privacy Violation	512	512	513	512	512	512	514	512	512	513	511	5635
Attempted User Privilege Gain	416	416	416	416	416	416	416	416	416	416	416	4576
Misc activity	317	317	317	317	317	317	317	317	317	317	317	3487
Attempted Information Leak	91	91	91	91	91	91	91	91	91	91	91	1001
A Network Trojan was detected	70	71	71	71	71	71	71	71	71	71	71	780
Attempted Administrator Privilege Gain	34	34	34	34	34	34	34	34	34	34	34	374
Potentially Bad Traffic	33	33	33	33	33	33	33	33	33	33	33	363
Not Suspicious Traffic	23	23	23	23	23	23	23	23	23	23	23	253
Detection of a Network Scan	18	18	18	18	18	18	18	18	18	18	18	198
Attempted Denial of Service	6	6	6	6	6	6	6	6	6	6	6	66
Generic Protocol Command Decode	4	4	4	4	4	4	4	4	4	4	4	44
Executable code was detected	1	1	1	1	1	1	1	1	1	1	1	11

Alerts on Signatures Published After Traffic

2m ago

Time	Event
7/14/15 3:36:00.661 PM	07/14/2015-15:36:00.661495 [**] [1:2812028:2] ETPRO POLICY Python Requests Suspicious User Agent [**] [Classification: Potential Corporate Privacy Violation] [Priority: 1] {TCP} 128.61.52.148:38242 -> 52.25.47.242:9090
7/14/15 3:36:00.661 PM	07/14/2015-15:36:00.661495 [**] [1:2812028:2] ETPRO POLICY Python Requests Suspicious User Agent [**] [Classification: Potential Corporate Privacy Violation] [Priority: 1] {TCP} 128.61.52.148:38242 -> 52.25.47.242:9090
7/14/15 3:36:00.661 PM	07/14/2015-15:36:00.661495 [**] [1:2812028:2] ETPRO POLICY Python Requests Suspicious User Agent [**] [Classification: Potential Corporate Privacy Violation] [Priority: 1] {TCP} 128.61.52.148:38242 -> 52.25.47.242:9090

Packet Capture Date

20150714

First Signature Date

20150715

Second Signature Date

20150716

Ruleset

ET PRO

1m ago

Time	Event
7/14/15 3:40:21.749 PM	07/14/2015-15:40:21.749394 [**] [1:2012141:2] ET POLICY Protocol 41 IPv6 encapsulation potential 6in4 IPv6 tunnel active [**] [Classification: Potential Corporate Privacy Violation] [Priority: 1] {IPV6} 143.215.50.118:0 -> 192.88.99.1:0
7/14/15 3:38:48.005 PM	07/14/2015-15:38:48.005754 [**] [1:2012141:2] ET POLICY Protocol 41 IPv6 encapsulation potential 6in4 IPv6 tunnel active [**] [Classification: Potential Corporate Privacy Violation] [Priority: 1] {IPV6} 143.215.50.118:0 -> 192.88.99.1:0
7/14/15 3:36:00.661 PM	07/14/2015-15:36:00.661495 [**] [1:2812028:2] ETPRO POLICY Python Requests Suspicious User Agent [**] [Classification: Potential Corporate Privacy Violation] [Priority: 1] {TCP} 128.61.52.148:38242 -> 52.25.47.242:9090